PHINS — AI & BI Optimization Review

PHINS — Investor-Lens AI & BI Optimization Review

Reviewer posture: Written as if by a peer "most advanced, high-tech insurance & savings platform" performing technical due diligence after reading the PHINS investor deck and the structured UML.

Scope of this document: Architecture / UML optimization findings, plus **PR-ready** AI and BI improvement specifications. **No platform code is changed by this document** — it is a review artifact. Every recommendation is written so it can be lifted into an implementation PR if PHINS decides to proceed, and every recommendation is gated by a **data-integrity guardrail** so that adding AI/BI intelligence never weakens the platform's core invariants.

Method: Direct read of the UML (`docs/uml/*.puml`, `docs/uml/rendered/*.svg`), the data architecture (`docs/platform_data_architecture.md`, `docs/health_marketplace_architecture.md`), and the implemented AI/BI/actuarial/ledger code in `services/`, `web_portal/`, `ai_automation_controller.py`, and `scheduler/`. This review intentionally separates **what the deck/UML promise** from **what the code does today**.

> **Implementation status (this PR):** A first wave of these recommendations is now **implemented** and test-backed (additive, integrity-preserving): **AI-1** append-only AI decision log (`services/ai_decision_log.py`), **AI-2** per-segment thresholds + recommend-only calibration (`services/ai_threshold_config.py`), **AI-3** model registry with rules fallback (`services/ai_model_registry.py`), the AI agent (`ai_automation_controller.py`) now logs every decision and resolves thresholds per segment, **BI-1** canonical KPIs (`services/kpi_definitions.py`), **BI-2** activated dashboard cache (`services/bi_analytics_service.py`), and the **DI-1** opt-in `PHINS_REQUIRE_DATABASE` fail-fast guardrail. Items marked *(shipped)* below are live; the rest remain PR-ready specs. The investor-facing summary is served on the pitch dashboard under "Investor Documents".
>
> **Note on prior assessments:** `PHINS_PLATFORM_ASSESSMENT.md` lists a CRITICAL "BI service is unimportable (`SyntaxError` line 30)" finding. That is now **stale** — `services/bi_analytics_service.py` compiles cleanly (`python3 -m compileall` passes) and the `/api/bi/*` endpoints import. This review supersedes that item and focuses on what remains.

1. Verdict (the one-paragraph version)

PHINS is **architecturally credible and intriguing**. The *target* design in the UML and the marketplace architecture doc is genuinely world-class: ledger-first, outbox-driven, idempotent, coverage-aware, with a feature store and decisioning service. The *implemented* platform has real substance in places investors underestimate (a coherent deterministic actuarial kernel; a hash-chained platform event ledger; thoughtful integrity validators incl. a marketplace foundation checker). The gap that matters for valuation is this: **the "AI" is presented as machine learning but is implemented as rule-based heuristics with no model registry, no feedback loop, and no ML runtime in the production image**, and the **BI layer recomputes every metric O(n) per request with a declared-but-unused cache and no precomputed snapshots**. Neither gap is fatal; both are **closeable with well-scoped PRs** that *reuse scaffolding already present*. The single most important thing to protect while closing them is **data integrity** — which is exactly why every recommendation below is integrity-gated.

If I were investing: I would fund it, conditioned on the AI-1/AI-2 and BI-1/BI-2 milestones below landing first, because they convert the platform's "AI/BI" narrative from *interface* to *substance* without large rewrites.

2. What I verified — credit where due

These are real, and they de-risk the investment:

Asset	Evidence	Why it matters
Deterministic actuarial stack	`services/actuarial_service.py` (mortality/disability tables, lapse, IFRS-17-style BEL/RA/CSM, IBNR %), `services/actuarial_valuation.py` (PVFP, required-capital, `integrity_hash`)	A defensible pricing/reserving core. Explainable by construction — a *regulatory feature*, not a bug.
Hash-chained platform event ledger	`services/platform_event_ledger_service.py` — `compute_entry_hash()`, `append_event()` with `sequence_no/previous_hash/entry_hash`, `reconcile_ledger_entries()`	A tamper-evident backbone exists. This is the right foundation to make canonical.
Layered integrity validators	`services/platform_integrity_service.py` incl. `validate_marketplace_foundation()` (wallet-hold coverage, settlement aging, markup recognition, payer-receivable aging, refund lineage)	The control vocabulary is already written down and partly enforced.
Ledger-first target architecture	`docs/health_marketplace_architecture.md`, `docs/uml/phins_platform_overview.puml` (Outbox Publisher, Idempotency Store, Reconciliation Jobs, Feature Store)	The destination is correctly drawn. The work is wiring, not invention.
Honest documentation in places	`ai_automation_controller.py` comments (`"In production, this would use trained ML models"`)	Engineering is not hiding the gap — it's labeled.

3. Architecture / UML optimization — gaps above and under the radar

The UML draws a clean target. The optimizations below are where the *drawing* and the *running system* diverge. I split them into **above the radar** (visible in any serious technical review) and **under the radar** (latent; only visible by reading the code paths).

3.1 Above the radar

- **A1 — The UML shows microservices; the runtime is one `BaseHTTPRequestHandler`.**

`web_portal/server.py` is a single ~48k-LOC module with `do_GET`/`do_POST` dispatchers measured in thousands of lines each. The component diagram (`phins_components` in `docs/uml/phins_platform_overview.puml`) lists ~20 services and an API Gateway that **do not exist as separable deployables**. *Optimization:* keep the monolith, but make the UML honest by marking which components are logical (in-process) vs physical, and continue the `web_portal/api_*.py` extraction pattern (the precedent already exists: `api_bi_analytics.py`, `api_delivery_bidding.py`). This is an architecture-doc + incremental-refactor item, not a rewrite.

- **A2 — The "Feature Store → Decisioning" arrow is aspirational.** The component UML wires `Feature Store → Decisioning Service (Actuary/AI)`. No feature store exists in code; `ai_automation_controller.py` reads request dicts directly. *Optimization:* either (a) annotate the UML as target-state, or (b) implement the minimal feature-snapshot table from AI-1 below, which is the smallest real thing that makes that arrow true.

- **A3 — Outbox / Idempotency are drawn and schema'd but not wired.**

`database/repositories/marketplace_repository.py` defines `OutboxRepository` and `IdempotencyRepository`; the UML shows `Outbox Publisher → Event Bus`. Grep of `web_portal/server.py` and `web_portal/api_extensions.py` finds **zero** call sites. The "idempotency everywhere" principle in `docs/health_marketplace_architecture.md` §4 is currently a principle, not a control. *Optimization:* DI-3 below.

3.2 Under the radar

- **U1 — Split-brain between in-memory globals and DB.**

`web_portal/server.py` rebinds module-level stores (`CUSTOMERS`, `POLICIES`, `CLAIMS`, `BILLING`) during DB recovery, while long-lived service singletons captured the *original* dict references. After a swap, the request path and the reporting/integrity path can read different worlds — and an integrity check can

return PASS against stale data. This is the **highest-value latent risk** and it directly threatens the "flawless data integrity" claim. (Mitigation in §6, guardrail DI-1.)

- **U2 — Parallel ledgers with no scheduled cross-reconciliation.**

At least four ledger-shaped stores coexist: the in-memory `TRANSACTION_LEDGER`, the DB wallet journal (`services/wallet_ledger_service.py`), the `NFT_LEDGER`, and JSON backups (`services/ledger_backup_service.py`). `platform_event_ledger_service` is the natural canonical log, but nothing **scheduled** reconciles the others against it. `scheduler/runner.py` runs **only** monthly auto-pay.

- **U3 — Actuarial simulations are process-global, not durable.**

Simulation snapshots live in `ACTUARIAL_SIMULATIONS: Dict[...]` inside `web_portal/server.py`. A restart loses them unless separately exported. For a platform whose differentiation is actuarial, the model outputs aren't first-class persisted artifacts.

- **U4 — Money is `float` far more often than `Decimal`.**

Only ~6 of ~74 service modules import `decimal`; ~54 use `float(...)` on monetary fields and `round(x, 2)` is pervasive (peak ~158 occurrences in `actuarial_service.py`). `accounting_engine.py` and `reserves_reporting_service.py` do it right with `Decimal`. Mixed money types are the classic source of slow reconciliation drift in financial systems.

- **U5 — Default integrity secret.**

`services/advanced_portfolio_integrity_service.py` falls back to a hardcoded HMAC key (`PHINS_INTEGRITY_2026`) when the env var is absent. A tamper-evidence control keyed by a public constant is not tamper-evident.

4. AI improvements — PR-ready specifications

Design philosophy (shared by all AI items): PHINS's rule-based decisioning is an *asset* for explainability and regulation. The goal is **not** to replace rules with an opaque model; it is to (1) **persist every decision** so the platform can learn, (2) **calibrate** thresholds from real outcomes, and (3) **optionally** layer a transparent, versioned model behind a registry — always keeping the rules as a deterministic fallback. This preserves auditability while unlocking the "AI" story.

Each item below is sized for a single PR and lists: the change, the files, the new schema (if any), acceptance criteria, and the **integrity guardrail**.

AI-1 — Persist every automated decision (the keystone)

- **Problem:** `AIAutomationController` makes underwriting/claims/fraud decisions

but only updates in-process counters; `reset_metrics()` discards history. There is no record of inputs → decision → human override. Without this, no learning, no audit, no calibration. The marketplace architecture already names the target entity: `model_decisions` (`docs/health_marketplace_architecture.md` §"Canonical data model").

- **Change:** Add an append-only `ai_decisions` table and a thin repository; write

one row from `AIAutomationController` after every `auto_underwrite` / `auto_process_claim` / `generate_auto_quote`, and a follow-up row (or update) when a human overrides.

- **Files:** `database/models.py` (new `AIDecision` model),

`database/repositories/ai_decision_repository.py` (new), `database/manager.py` (expose `.ai_decisions` property), `ai_automation_controller.py` (write-through after each decision), the route(s) that record human underwriting/claims actions in `web_portal/server.py` (write the override row).

- **Schema (illustrative):**

```
# database/models.py
class AIDecision(Base):
    __tablename__ = "ai_decisions"
    id = Column(String, primary_key=True)            # AIDEC-...
    decision_type = Column(String, nullable=False)   # quote|underwrite|claim|fraud
    entity_type = Column(String)                     # policy|claim|customer
    entity_id = Column(String, index=True)
    inputs_json = Column(JSON, nullable=False)       # feature snapshot at decision time
    output_json = Column(JSON, nullable=False)       # decision, score, thresholds, reason_codes
    model_version = Column(String, nullable=False)   # "rules-v1" today; registry id later
    confidence = Column(Float)
    created_at = Column(DateTime, default=utcnow)
    human_override = Column(String)                  # null until a human disagrees
    override_reason = Column(Text)
    overridden_by = Column(String)
    overridden_at = Column(DateTime)
```

- **Acceptance:** every call to the controller produces exactly one immutable

decision row; overrides are linked, never destructive; a new test asserts row count == decisions made; existing tests still pass.

- **Integrity guardrail (DI):** rows are **append-only** (no UPDATE of

`inputs_json`/`output_json`; overrides are *new fields or new linked rows*). The decision log is reporting/training data and **must not** feed back into a money posting without a human or rule gate. Effort: **Low.**

AI-2 — Weekly threshold calibration from outcomes

- **Problem:** A single global `auto_approve_threshold = 0.85`

(`ai_automation_controller.py`) cannot be correct across segments (a 25-y/o office worker vs a 60-y/o construction worker). There is no mechanism to adjust it from results.

- **Change:** A scheduled job that reads `ai_decisions` (AI-1), computes

precision/recall of auto-decisions against human overrides **per segment** (age band × occupation × product), and writes recommended thresholds to a `ai_thresholds` config row. The controller loads thresholds at startup; if the table is empty it uses today's constants (safe default).

- **Files:** `scheduler/runner.py` (add a calibration entry point — today it only

wraps `run_monthly_auto_pay`), a new `scripts/run_ai_calibration.py`, `render.yaml` (add a weekly cron alongside `phins-monthly-auto-pay`), `ai_automation_controller.py` (load thresholds; keep constant fallback).

- **Acceptance:** with seeded decisions+overrides, the job emits per-segment

thresholds; controller behavior changes only when a threshold row exists; calibration is **recommend-only** until a human promotes it (config flag).

- **Integrity guardrail:** calibration reads the append-only log and writes

configuration, never financial state. Promotion of new thresholds is gated by an explicit admin action + audit row. Effort: **Low–Medium.**

AI-3 — Transparent, versioned model behind a registry (rules stay as fallback)

- **Problem:** Once AI-1 accrues data, PHINS can fit a **transparent** model

(logistic regression / GLM — the actuarially-accepted family) without losing explainability. Today there is no model artifact, no registry, and the production image ships **no ML libraries** (no numpy/pandas/sklearn/scipy/ statsmodels in `requirements.txt`).

- **Change:** Add a `services/ai_model_registry.py` that loads named, versioned

artifacts (start with `joblib` on disk; later S3 via the already-present `boto3`). The controller asks the registry for a model; if none is present it uses the rule-based scorer (unchanged). Training lives **outside** the request path (a script/notebook), and the model file is an artifact, not code.

- **Files:** `requirements.txt` (add `scikit-learn`, `joblib`, `numpy` — scoped,

with a note that the production image grows), `services/ai_model_registry.py` (new), `ai_automation_controller.py` (registry lookup + fallback), `database/models.py` (`ModelVersion` row — already named in the marketplace doc).

- **Acceptance:** with no artifact, behavior is byte-identical to today (rules);

with an artifact, the decision row records `model_version` = the registry id and

the rule-based score is still computed and logged for comparison.

- **Integrity guardrail:** the model **scores**, it does not move money. Every

model decision still writes the AI-1 append-only row including the rules-vs-model

delta, so drift is observable. Rollback = point the registry at the prior

version. Effort: **Medium.**

AI-4 — LLM-assisted triage for free-text (summarize, never decide)

- **Problem:** `services/claims_bot_service.py` and `ai_risk_reports_service.py`

do keyword/regex extraction on narratives. An LLM is well-suited to

summarization and *initial categorization* of free-text claim descriptions

and uploaded documents.

- **Change:** Behind a feature flag and an optional dependency, add an LLM call

that produces a **summary + suggested category + extracted entities** attached

to the claim/report as advisory metadata. **Approval/payment decisions remain

deterministic** (rules/registry).

- **Files:** `services/claims_bot_service.py` (advisory enrichment only),

config/env for provider + key (no secrets in code), a guard that degrades

gracefully when the key/lib is absent.

- **Acceptance:** with no key configured, the system behaves exactly as today;

with a key, claims gain an advisory summary that does **not** alter the

computed decision; PII handling documented.

- **Integrity guardrail:** LLM output is **advisory metadata only** — it can never

set `approved_amount`, change a status, or post a ledger entry. Effort: **Medium.**

AI-5 — Make actuarial simulation outputs durable & versioned

- **Problem (U3):** `ACTUARIAL_SIMULATIONS` is a process global; restarts lose it;

`actuarial_valuation.py` already computes an `integrity_hash` that is never

stored.

- **Change:** Persist each simulation snapshot + its valuation + `integrity_hash`

to the existing `ActuarialRepository` (or a small new `actuarial_simulations`

table), keyed by `simulation_id` and `table_version`.

- **Files:** `database/models.py`, `database/repositories/actuarial_repository.py`,

the simulation creation path in `web_portal/server.py`.

- **Acceptance:** a simulation survives a restart; re-loading a snapshot

re-verifies its `integrity_hash`; BI/actuarial dashboards can reference a stored run rather than recompute.

- **Integrity guardrail:** snapshots are **immutable** once written

(hash-verified on read). Effort: **Medium.**

5. BI improvements — PR-ready specifications

Design philosophy: Today every dashboard recomputes from full live state on each request, with `self.cache`/`cache_ttl_seconds` declared but **never used** (`services/bi_analytics_service.py`). The path to "real BI" is: (1) stop paying O(n) on every request, (2) precompute snapshots on a schedule, (3) make integrity a continuous time series rather than a fire-on-demand check, and (4) define each KPI **once**. None of this requires leaving the current stack; an external BI tool (e.g. Omni) is an option once events flow, but is not a prerequisite.

BI-1 — Define each KPI in exactly one place

- **Problem:** "loss ratio" is computed in at least three places with subtly

different denominators (`bi_analytics_service.py`, `services/financial_reporting_service.py`, `services/reserves_reporting_service.py`). Investors and regulators will ask "which number is true?" — and today there are several.

- **Change:** Create `services/kpi_definitions.py` with one canonical function per

KPI (loss ratio, MRR/ARR, approval rate, receivables ratio, health scores) and import it everywhere. No behavior change beyond convergence on one definition.

- **Acceptance:** all callers import the canonical function; a test pins each KPI's

formula; the executive dashboard number equals the financial-reporting number.

- **Integrity guardrail:** pure functions over passed-in data; no state mutation.

Effort: **Low.**

BI-2 — Activate the dead cache

- **Problem:** `BIAnalyticsService.__init__` declares `self.cache` and

`self.cache_ttl_seconds = 300` and never uses them; every `/api/bi/*` request recomputes from scratch.

- **Change:** Wrap each `get_*_analytics` / `get_executive_dashboard` in a small

`_cached(key, ttl, fn)` helper using the existing fields. Cache key includes a cheap state fingerprint (e.g. counts + max-updated-at) so stale data is bounded.

- **Files:** `services/bi_analytics_service.py` only.
- **Acceptance:** repeated dashboard calls within TTL return identical payloads

without re-iterating state; a test asserts the underlying compute runs once within the window; cache invalidates when the fingerprint changes.

- **Integrity guardrail:** cache is **read-only derived data**; on any doubt it

recomputes. The fingerprint guarantees a cached dashboard never contradicts a changed store. Effort: **Low.**

BI-3 — Hourly precomputed dashboard snapshots

- **Problem:** Even cached, the first request after each change pays O(n). For

investor-grade dashboards you want sub-second reads always.

- **Change:** A scheduled job computes the executive/customer/supplier/delivery

dashboards and writes them as timestamped snapshot rows; the API serves the latest snapshot (with a "computed_at" badge) and falls back to live compute if no snapshot exists.

- **Files:** `scheduler/runner.py` + a new `scripts/run_bi_snapshot.py`,

`render.yaml` (hourly cron), a small `bi_snapshot` table/repository, `web_portal/api_bi_analytics.py` (serve snapshot, fallback to live).

- **Acceptance:** dashboards read a snapshot in O(1); a time series of snapshots

accumulates (enabling trend charts the deck implies but can't currently draw); fallback path covered by a test.

- **Integrity guardrail:** snapshots are **immutable, timestamped** derived views;

they never become a write source. Effort: **Medium.**

BI-4 — Continuous integrity as a time series (not fire-on-demand)

- **Problem:** Integrity is only checked when someone calls

`/api/integrity/validate`. There is no history, so "were we consistent last Tuesday?" is unanswerable, and the request-time cost is O(n^2) on some cross-pipeline checks.

- **Change:** Schedule `validate_all()` (+ `validate_marketplace_foundation()`)

and store each run as a `validation_run` row; the API serves the latest run and a trend. Add inverted indexes (customer→policy, policy→claim) built once per run to kill the O(n^2) cross-checks.

- **Files:** `scheduler/runner.py` + `scripts/run_integrity_sweep.py`,

`services/platform_integrity_service.py` (optional index precompute),

a `validation_run` table/repository, `web_portal/api_bi_analytics.py`.

- **Acceptance:** integrity runs on schedule; the API returns the latest result + a pass/fail time series; per-run errors/warnings are queryable.
- **Integrity guardrail:** this *strengthens* integrity — it turns a manual check into a monitored, historized control. Effort: **Medium.**

BI-5 — Canonical event emission for BI/AI (and future external BI)

- **Problem:** The marketplace doc specifies a versioned event vocabulary (`order.created`, `claim.adjudicated`, `refund.completed`, …); BI/AI consume live state instead of events, so analytics and the ledger can drift.
- **Change:** Emit the canonical events into the existing `platform_event_ledger_service` (already hash-chained) at the same point the underlying write happens, and have BI snapshots/AI features read the event stream. This is the bridge that later makes an external BI tool (Omni/Databricks) a drop-in, but delivers value immediately in-process.
- **Files:** the write points in `web_portal/server.py`, `services/platform_event_ledger_service.py` (canonical event types).
- **Acceptance:** strategic actions emit one versioned event each; BI/integrity can be derived from the event stream; chain reconciliation still passes.
- **Integrity guardrail:** events are appended to the **hash-chained** log; emission shares the OLTP transaction (this is also the on-ramp to DI-3's outbox). Effort: **Medium.**

6. Data integrity — keeping the "flawless" claim true

The platform *markets* flawless data integrity. The infrastructure to be flawless exists; the wiring does not fully deliver it yet. These items are **prerequisites and guardrails** — none of the AI/BI work above should ship without DI-1 in place, because an AI/BI layer reading a split-brain store would *amplify* the problem.

- **DI-1 — Eliminate split-brain (U1).** Replace the global dict rebinding in `web_portal/server.py` with a stable store handle (e.g. `CustomerStore` exposing `get/set/items`) that proxies in-memory until DB is live, then proxies DB — **services hold the store, not the dict**. Alternative: fail-fast when `USE_DATABASE=true` and DB is unavailable (a `PHINS_REQUIRE_DATABASE=1` hard exit) rather than silently falling back. *This is the single change that makes the integrity story true* and must precede AI-1/BI-2.

- **DI-2 — One canonical event log (U2).** Make

`platform_event_ledger_service` authoritative; derive wallet balances, supplier ledgers, and the balance sheet from it; reduce `ledger_backup_service` to incremental snapshots keyed by max `sequence_no`. Add a **scheduled cross-ledger reconciliation** (extends BI-4).

- **DI-3 — Wire Outbox + Idempotency (A3).** Use the existing

`OutboxRepository`/`IdempotencyRepository` at the same point as the underlying write so event publication can't diverge from the OLTP commit, and `do_POST` retries are safe. The marketplace doc's "idempotency everywhere" becomes real.

- **DI-4 — Money discipline (U4).** Introduce `services/_money.py` (Decimal-backed

`Money`), require financial services to use it, and add a CI lint blocking `round(x, 2)` in `services/billing*` / `accounting*`. Integrity validators flag any non-zero sub-cent difference.

- **DI-5 — Remove default integrity secret (U5).** Refuse to start (or refuse to

treat snapshots as tamper-evident) when the HMAC key in `advanced_portfolio_integrity_service.py` is the default constant.

Guardrail summary for AI/BI work: derived layers (AI scores, BI dashboards, caches, snapshots) are **read-only views** over the canonical, hash-chained, append-only state. They may *recommend*; they may never *post*. Money moves only through the existing ledger + accounting engines, under rule/human gates, with an append-only decision record. Adhering to this keeps integrity flawless **while** adding intelligence.

7. PR-ready backlog (rank-ordered, each shippable in isolation)

Ordered by value-to-effort, with the integrity prerequisite first. Suggested branch names follow the repo convention (`cursor/<name>-ac1a`).

#	Item	Type	Effort	Integrity-safe?	Suggested branch
1	**DI-1** Require-DB fail-fast guardrail *(shipped)* + full store indirection *(spec)*	Integrity (prereq)	Medium	Makes integrity true	`cursor/di-store-handles-ac1a`
2	**BI-1** One canonical KPI module *(shipped)*	BI	Low	Yes (pure fns)	`cursor/bi-canonical-kpis-ac1a`
3	**BI-2** Activate dashboard cache *(shipped)*	BI	Low	Yes (read-only)	`cursor/bi-activate-cache-ac1a`

#	Item	Type	Effort	Integrity-safe?	Suggested branch
4	**AI-1** Persist every AI decision *(shipped)*	AI (keystone)	Low	Yes (append-only)	`cursor/ai-decision-log-ac1a`
5	**BI-4** Continuous integrity + history	BI/Integrity	Medium	Strengthens it	`cursor/bi-integrity-timeseries-ac1a`
6	**AI-2** Weekly threshold calibration *(shipped: per-segment thresholds + recommend-only calibration)*	AI	Low–Med	Yes (config only)	`cursor/ai-threshold-calibration-ac1a`
7	**BI-3** Hourly dashboard snapshots	BI	Medium	Yes (immutable)	`cursor/bi-dashboard-snapshots-ac1a`
8	**DI-4** Decimal money discipline	Integrity	Medium	Yes	`cursor/money-decimal-discipline-ac1a`
9	**BI-5** Canonical event emission	BI/AI bridge	Medium	Yes (hash-chained)	`cursor/canonical-event-emission-ac1a`
10	**DI-3** Wire Outbox + Idempotency	Integrity	Medium	Yes	`cursor/wire-outbox-idempotency-ac1a`
11	**AI-5** Durable actuarial simulations	AI	Medium	Yes (immutable)	`cursor/actuarial-sim-persistence-ac1a`
12	**AI-3** Versioned model + registry *(shipped: registry + rules fallback; training/artifacts next)*	AI	Medium	Yes (scores only)	`cursor/ai-model-registry-ac1a`
13	**AI-4** LLM advisory triage	AI	Medium	Yes (advisory only)	`cursor/ai-llm-triage-ac1a`
14	**DI-2** Single canonical ledger	Integrity	High	Yes	`cursor/canonical-event-ledger-ac1a`

8. Implementation status & integrity stance

This document began as a review; it now ships alongside a **first wave of implementation** (see the *(shipped)* rows in §7 and the served *AI & BI Implementation Summary*). The implemented changes are deliberately **additive and integrity-preserving**:

- **AI agent** (`ai_automation_controller.py`) now records every quote / underwriting / claim decision to an **append-only** log (`services/ai_decision_log.py`); human overrides are linked additively and never rewrite the original. Underwriting thresholds resolve **per segment** (`services/ai_threshold_config.py`) with defaults identical to the prior global

constants, and a model registry (`services/ai_model_registry.py`) is consulted with the **deterministic rules remaining authoritative** (no artifacts ship, so behavior is unchanged).

- **BI** uses one **canonical KPI module** (`services/kpi_definitions.py`) and an **activated, content-fingerprinted cache** that recomputes the moment inputs change, so a cached dashboard can never contradict live state.
- **Data integrity** gains an opt-in `PHINS_REQUIRE_DATABASE` fail-fast (default off) so the platform can refuse to run on the volatile in-memory fallback.

The remaining items in §4–§6 stay as PR-ready specs. The governing invariant is unchanged and was preserved throughout: **every balance and every decision is reproducible from append-only, hash-chained state, and derived AI/BI intelligence may recommend but never post.** Money continues to move only through the existing ledger and accounting engines under rule/human gates.